Xi'an Jiaotong-Liverpool University
西交利物浦大學

Transactions and Recovery

Yu Liu (Yu.Liu02@xjtlu.edu.cn)

This Lecture

- JDBC Basics
- Transactions
  - Example using JDBC
- Database security and recovery
  - System and Media Failures

JDBC

Connecting your Java application to a database

What is JDBC?

- **Definition**: JDBC (Java Database Connectivity) is the Java API that manages connecting to a database, issuing queries and commands, and handling result sets obtained from the database.

- **Usage**: To use JDBC in a Java application, you need the JDBC connector first.
  - Download for MariaDB: https://downloads.mariadb.org/connector-java/
  - Then, Add it into your Java project.

Connecting to Databases

- You need a Connection class

```
public static void main(String[] args) {
    Connection conn = null;
    try {
        conn = DriverManager.getConnection(csseSQLURL);
        System.out.println("Connected to database");
    } catch (SQLException e) {
        System.out.println(e.getMessage());
    }
}
```

- The URL in getConnection() should be pointing to your database server. Example:

```
"jdbc:mysql://csse-mysql.xjtlu.edu.cn:3306/jianjunchen?user=JianjunChen&password=secret"
```

Executing Queries

- Use your `Connection` object to get a `Statement` object:

```
Statement statement = null;
try {
    statement = conn.createStatement();
} catch (SQLException e) {
    System.out.println(e.getMessage());
}
```

- Then execute any queries you want:

```
statement.execute("CREATE TABLE myTable (id INT PRIMARY KEY");
for (int i = 1; i < 4; i++) {
  statement.execute("INSERT INTO myTable VALUES (" + i + ")");
}
```

Getting Results from Queries

- After executing a SELECT statement, you can get its result:

```
statement.execute("SELECT * FROM myTable;");
ResultSet rs = statement.getResultSet();
while (rs.next()) {
    System.out.println("The id is: " + rs.getInt("id"));
}
```

- A query result may contain multiple rows.
  - When “rs.next()” is called the first time, the pointer points to 1 and returns true.
  - The next time it is called, it points to 2 and returns true again.
  - The third time it is called, it points to 3 and returns true.
  - In the forth time, it returns false.

id
1
2
3

- You must call “rs.next()” at least once to obtain values!

Transactions

ACID properties

Transaction in SQL

The Concurrency Problem

- Check the code in `SQLDemo`. Adjust the value of private member `csseURL` so that it uses your own account.

```
public class SQLDemo {
    // You can replace this with your own MySQL account
    5 usages
    private static String csseURL = "jdbc:mysql://csse-my
```

- Comment out `demoTransaction` and uncomment `demoNoTransaction` in `main`.

```
public static void main(String[] args) {
    try {
        demoBasicSQL();
        Thread.sleep( millis: 1000);
        demoNoTransaction();
        //demoTransaction();
    } catch (SQLException | InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

The Concurrency Problem

- **Logic of the Demo**: repeatedly add salary to one staff and subtract salary from another staff.

```
for (int i = 0; i < 200; i++) {
    statement.execute( sql: "update staff set salary = salary + 50 where staffID = 1000");
    statement.execute( sql: "update staff set salary = salary - 50 where staffID = 2000");
}
```

- At the same time, get the sum of these staff members' salaries.

```
statement.execute( sql: "SELECT salary FROM staff WHERE staffID in (1000,2000)");
ResultSet rs = statement.getResultSet();
rs.next();
int s1 = rs.getInt( columnLabel: "salary");
rs.next();
int s2 = rs.getInt( columnLabel: "salary");
System.out.println("Sum of 2 salaries: " + (s1 + s2));
```

The Concurrency Problem

- **Ideal output**: Since the sum of two salaries is not increased, the program should consistently output 2000.
- **Actual output**: Actual value printed is sometimes 2050.

```
Staff ID 4000: He works in branch
--------------------------------
Sum of 2 salaries: 2000
Sum of 2 salaries: 2000
Sum of 2 salaries: 2050

Process finished with exit code 0
```

Bad Timing

- The reason is that the SELECT statement is sometimes executed right after the salary increase.

```
for (int i = 0; i < 200; i++) {
    statement.execute( sql: "update staff set salary = salary + 50 where staffID = 1000");
    statement.execute( sql: "update staff set salary = salary - 50 where staffID = 2000");
}

statement.execute( sql: "SELECT salary FROM staff WHERE staffID in (1000,2000)");
```

Time	salary update	sum up
1	+50	
2	-50	
3		sum
4	+50	
5	-50	

Example of good timing

Time	salary update	sum up
1	+50	
2	-50	
3	+50	
4		sum
5	-50	

Example of bad timing

Transactions: Definition

- **Definition**: A transaction is an action, or a series of actions, carried out by a single user or an application program, which reads or updates the contents of a database.
- A transaction is a 'logical unit of work' on a database
  - Each transaction does something in the database
  - No part of it alone achieves anything of use or interest

- Transactions are the unit of recovery, consistency, and integrity as well

Transactions - **ACID** properties

- **Atomicity**: Either the entire transaction takes place at once or doesn't happen at all (cannot occur partially).

- **Consistency**: Integrity constraints must be maintained so that the database is consistent before and after the transaction

- **Isolation**: Multiple transactions can occur concurrently without leading to the inconsistency of the database state

- **Durability**: once a transaction finishes, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs

Example of transaction

Transfer £50 from account A to account B:

1. Read(A)
2. A = A - 50
3. Write(A)
4. Read(B)
5. B = B+50
6. Write(B)

- **Atomicity** - shouldn't take money from A without giving it to B
- **Consistency** - money isn't lost or gained
- **Isolation** - other queries shouldn't see A or B change until completion
- **Durability** - the money does not go back to A

The Transaction Manager

The <u>transaction manager</u> enforces the ACID properties

- It schedules the operations of transactions
- COMMIT and ROLLBACK are used to ensure atomicity
- Locks are used to ensure consistency (partially) and isolation for concurrent transactions
- A log is kept to ensure durability in the event of system failure

Transaction Support In MySQL

- **keywords and syntax**:

```
START TRANSACTION
    [transaction_characteristic [, transaction_characteristic] ...]

transaction_characteristic: {
    WITH CONSISTENT SNAPSHOT
  | READ WRITE
  | READ ONLY
}

BEGIN [WORK]
COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
SET autocommit = {0 | 1}
```

COMMIT and ROLLBACK

- COMMIT signals the successful end of a transaction
  - Any changes made by the transaction should be saved
  - These changes are now visible to other transactions
  - Writes the data change in the memory to the DB file.

- ROLLBACK signals the unsuccessful end of a transaction
  - Any changes made by the transaction should be undone
  - It is now as if the transaction never existed

Transaction Example

- **Example**:

```sql
START TRANSACTION;
SELECT @A:=SUM(salary) FROM table1 WHERE type=1;
UPDATE table2 SET summary=@A WHERE type=1;
COMMIT;
```

- More information can be found at :

https://dev.mysql.com/doc/refman/8.0/en/commit.html

Fixing the SQLDemo

- We can fix the concurrency problem of SQL Demo by using transaction.
- Comment out `demoNoTransaction` and uncomment `demoTransaction` in `main`.

```
public static void main(String[] args) {
    try {
        demoBasicSQL();
        Thread.sleep( millis: 1000);
        //demoNoTransaction();
        demoTransaction();
    } catch (SQLException | InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

Fixing the SQLDemo

- The new updater uses transaction

```
statement.execute( sql: "start transaction ");
for (int i = 0; i < 200; i++) {
    statement.execute( sql: "update staff set salary = salar
    statement.execute( sql: "update staff set salary = salar
}
statement.execute( sql: "commit");
conn.close();
```

- The problem is fixed:

```
-----------------------------------
Some of 2 salaries: 2000
Some of 2 salaries: 2000
Some of 2 salaries: 2000

Process finished with exit code 0
```

Data Security and Recovery

System failures and prevention methods

Data protection act 2018

Data Protection Act

- **DPA**: “A legal basis for handling of information in the United Kingdom (UK) relating to people living within”.

- Extended reading: DPA 1998 document uploaded on LMO.
  - DPA 2018 is a more recent version.

Principles to Follow

Other regulations like CCPA or GDPR also exist, but the general principles are:

1. **Data minimization**: Only collect what you need. Avoid collecting sensitive data (e.g., SSNs, biometrics) unless absolutely required.
2. **Purpose limitation**: Use data only for the stated purpose.
3. **Storage limitation**: Don’t keep data longer than necessary.
    1. Auto-delete accounts after inactivity.
    2. Retain transaction logs for a defined period only.
4. **Integrity and confidentiality**: Keep data secure and accurate.
5. **Transparency**: Let users know what data you collect and why.
6. **User control**: Let users access, update, or delete their data.

Data Security

- Privacy of users is important! User authentication information should also be encrypted or hashed.
  - SQL supports many encryption functions: https://dev.mysql.com/doc/refman/9.0/en/encryption-functions.html
- Example: passwords

```
INSERT INTO ... VALUES ('Jason', MD5('123'));
```

123 → MD5 → 202cb962ac59075b964b07152d234b70

- Easy to convert 123 to MD5 checksum, but much harder to convert in a reverse way.
  - MD5 is a kind of **hash** function. (https://www.geeksforgeeks.org/dsa/hash-functions-and-list-types-of-hash-functions/)

Access Control

- To further enhance the security of data access, one can adapt **role-based access control (RBAC)**.

- Separate roles for:
  - Admins
  - Analysts
  - Application services

- Never allow direct access to production databases for non-essential personnel.

Recovery

- Transactions should be durable, but we cannot prevent all sorts of failures:
  - System crashes
  - Power failures
  - Disk crashes
  - User mistakes
  - Sabotage
  - Natural disasters

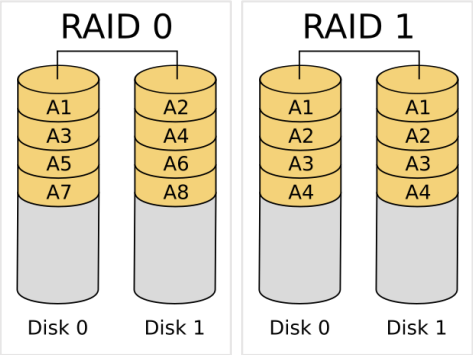
- Prevention is better than cure
  - Reliable OS
  - Security
  - UPS and surge protectors
  - RAID arrays
- However, we cannot protect against everything

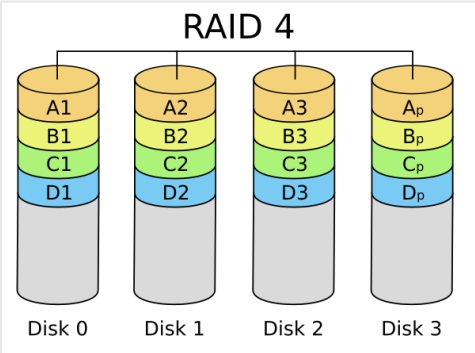
Media Failures

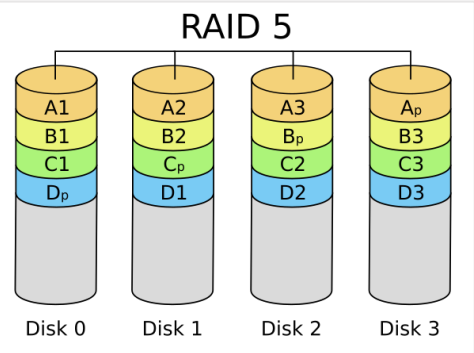
- System failures are not too severe
  - Only information since the last checkpoint is affected
  - This can be recovered from the transaction log.
- Media failures (disk crashes etc.) are more serious
  - The data stored to disk is damaged
  - The transaction log itself may be damaged

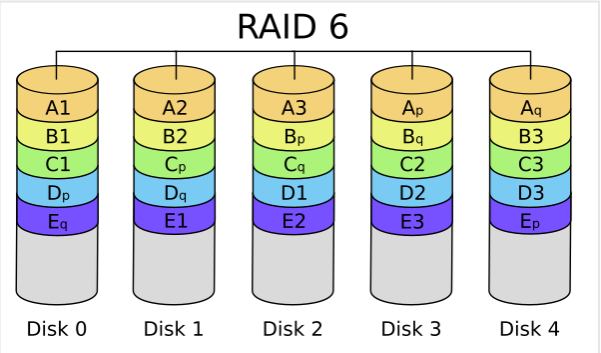
Media Failure Prevention: RAID

- **RAID**: Redundant Array of Independent Disks.
  - Widely used in servers.
  - Employ the techniques of striping, mirroring, or parity to create large reliable data stores from multiple hard disk drives (HDDs)
- **Common RAID types**:
  - RAID 0 (Stripping)
  - RAID 1 (Mirroring)
  - RAID 4 (striping with a dedicated parity disk)
  - RAID 5 (distributed parity)
  - RAID 6 (dual parity)



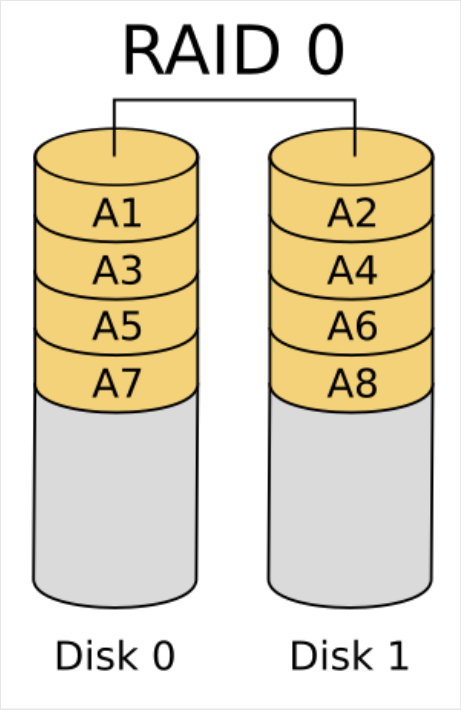






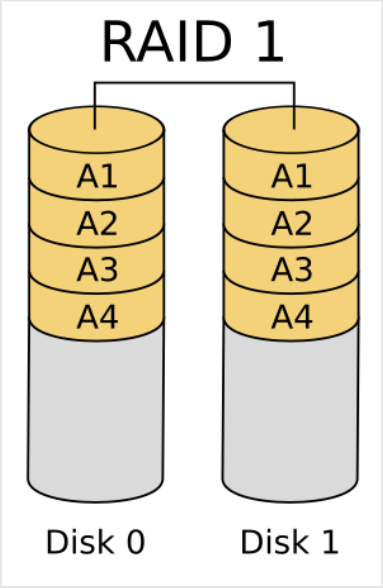
RAID 0

- **Stripping**: Splits data into blocks and spreads them across two or more disks.
- **Benefits**:
  - **Increased speed**: Each disk stores part of the data, allowing for faster read and write operations since data is read/written in parallel.
  - **Capacity**: The total storage capacity is the sum of all disk sizes.
- **Drawbacks:**
  - **Worse fault tolerance**: Data is lost if a single disk in the array breaks.
  - **Higher risk of failure**: More disks in RAID 0, the greater the risk of failure.



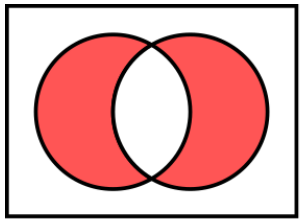
RAID 1

- **Mirroring**: Data is duplicated across two or more disks. Each write operation is written to every disk in the array.
- **Benefits**:
  - **Fault tolerance**: If one disk fails, the data is still available on the other disk(s).
  - **Read performance**: If there are multiple mirrors, reads can be distributed, improving read performance in some cases.
  - **Business continuity**: RAID 1 allows for continued database operations even during a disk failure.
  - **Fast recovery**: A failed disk can be replaced while the array continues to operate.
- **Drawbacks**:
  - **Capacity**: The storage capacity is limited to the size of the smallest disk in the array.
  - **Write performance**: Write speeds are typically the same as a single disk because each write operation is duplicated.



Parity using XOR

- **Parity bit in RAID 5**: Calculated using XOR (exclusive OR).



A	B	A XOR B
F	F	F
F	T	T
T	F	T
T	T	F

- **Example**: 0b01000100 XOR 0b11101111
  - **0b**XXXXX means this is a binary number.

	0	1	0	0	0	1	0	0
xor	1	1	1	0	1	1	1	1
	1	0	1	0	1	0	1	1

Parity using XOR

- **Current Data**:
  - 0b01000100 and 0b11101111 are your original data.
  - 0b10101011 is your parity data.

- **Recovery**: If any one of these data is missing, you can recover it using the others.

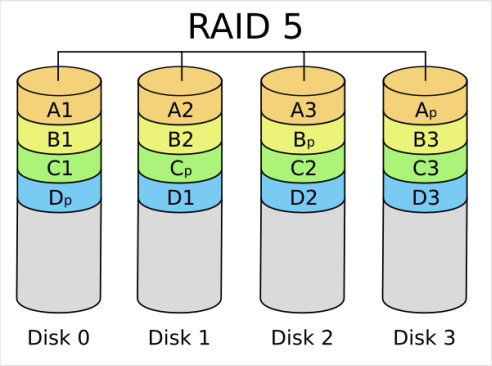
- **Example**: when data 0b11101111 is damaged.

	0	1	0	0	0	1	0	0
xor	1	0	1	0	1	0	1	1
	1	1	1	0	1	1	1	1

Parity: RAID 5 Example

- **XOR in RAID 5**: When multiple pieces of data are involved, XOR still works.
- **Example**: highlighted data are damaged

data 1	data 2	data 3	parity
1001	0110	1010	0101
1100	1011	0001	0110



- **Restoration**:

	1	0	0	1
xor	0	1	1	0
	1	1	1	1
xor	0	1	0	1
	1	0	1	0

	1	1	0	0
xor	0	0	0	1
	1	1	0	1
xor	0	1	1	0
	1	0	1	1

Media Failure Recovery: Backups

- **Backups**: Sometimes media failures and system failures cannot be recovered. Backups are needed to restore the system to a previous state.

- **Backup storages**:
  - External hard drives
  - Tape drives
  - Optical storage: resistant to water (flood proof)
- **Enhance security**:
  - Encrypt backups.
  - Store them securely and limit access.

Media Failure Recovery: Backups

- **Issue:**
  - The transaction log and entire contents of the database is written to secondary storage (often tape).
  - Time consuming, and often requires down time.
- **Backups frequency**
  - Frequent enough that little information is lost
  - Not so frequent as to cause problems
  - Every day (night) is common
- **Recovery**
  1. Restore the database from the last backup
  2. Use the transaction log to redo any changes made since the last backup
- If the transaction log is damaged, you can't do step 2
  - Store the log on a separate physical device to the database
  - The risk of losing both is then reduced

MQ

MQ